

4주 차 보고서

(2025년 1 월 18 일 ~ 2026년 1 월 24 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

LockFreeStack과 LockFreeQueue를 직접 만들어 보며 Lock-Free에 대한 이해도를 높였다. 아직 공부를 해도 잘 이해가 되지 않는 부분이라 나중에 적용해보면서 다시 한번 공부를 한다면 그때는 이해할 수 있을 것 같다.

```
void Push(const T& value)
{
    Node* node = new Node(value);
    node->next = _head;
    //if (_head == node->next)
    //{
    //    _head = node;
    //    return true;
    //}
    else
    {
        node->next = _head;
        return false;
    }
    while (_head.compare_exchange_weak(node->next, node) == false)
    {
    }
    // 이 사이에 새끼가 있냐?
    _head = node;
    // 1) head 쓰기
    // 2) head->next 쓰기
    // 3) head = head->next
    // 4) data 주 줄까지 읽음
    // 5) 주 줄까지 쓰기
    // 1) head 쓰기
    // 2) head->next 쓰기
    // 3) head = head->next
    // 4) data 주 줄까지 읽음
    // 5) 주 줄까지 쓰기
    bool TryPop(T& value)
    {
        Node* oldHead = _head;
        while (oldHead && _head.compare_exchange_weak(oldHead, oldHead->next) == false)
        {
        }
        if (oldHead == nullptr)
        {
            return false;
        }
        value = oldHead->data;
        // 잠시 삭제 보류
        delete oldHead;
        return true;
    }
}

bool TryPop(T& value)
{
    ++_popCount;
    Node* oldHead = _head;
    while (oldHead && _head.compare_exchange_weak(oldHead, oldHead->next) == false)
    {
    }
    if (oldHead == nullptr)
    {
        --_popCount;
        return false;
    }
    value = oldHead->data;
    TryDelete(oldHead);
    return true;
}

// 1) 데이터 분리
// 2) Count 체크
// 3) 나 혼자만 삭제
void TryDelete(Node* oldHead)
{
    // 나 외에 누가 있는지를 체크
    if (_popCount == 1)
    {
        // 나 혼자네?
        // 이왕 혼자인거, 삭제 예약된 다른 데이터들도 삭제해보자
    }
}

void TryDelete(Node* oldHead)
{
    // 나 외에 누가 있는지를 체크
    if (_popCount == 1)
    {
        // 나 혼자네?
        // 이왕 혼자인거, 삭제 예약된 다른 데이터들도 삭제해보자
    }
}

template<typename T>
class LockFreeStack
{
public:
    struct Node
    {
        Node(const T& value) : data(value), next(nullptr)
        {
        }
        T data;
        Node* next;
    };
    void Push(const T& value)
    {
        Node* node = new Node(value);
        node->next = _head;
        while (_head.compare_exchange_weak(node->next, node) == false)
        {
        }
        _head = node;
    }
    bool TryPop(T& value)
    {
        ++_popCount;
        Node* oldHead = _head;
        while (oldHead && _head.compare_exchange_weak(oldHead, oldHead->next) == false)
        {
        }
        if (oldHead == nullptr)
        {
            return false;
        }
        value = oldHead->data;
        // 잠시 삭제 보류
        delete oldHead;
        return true;
    }
}

void TryDelete(Node* oldHead)
{
    // 나 외에 누가 있는지를 체크
    if (_popCount == 1)
    {
        // 나 혼자네?
        // 이왕 혼자인거, 삭제 예약된 다른 데이터들도 삭제해보자
    }
}

void ChainPendingNodeList(Node* first, Node* last)
{
    last->next = _pendingList;
    while (_pendingList.compare_exchange_weak(last->next, first) == false)
    {
    }
}
```

```

void ChainPendingNodeList(Node* node)
{
    Node* last = node;
    while (last->next)
        last = last->next;

    ChainPendingNodeList(node, last);
}

void ChainPendingNode(Node* node)
{
    ChainPendingNodeList(node, node);
}

static void DeleteNodes(Node* node)
{
    while (node)
    {
        Node* next = node->next;
        delete node;
        node = next;
    }
}

private:
    atomic<Node*> _head;

    atomic<uint32> _popCount = 0; // Pop을 실행중인 스레드 개수
    atomic<Node*> _pendingList; // 삭제되어야 할 노드들(첫번째 노드)
};

shared_ptr<T> TryPop()
{
    CountedNodePtr oldHead = _head;
    while (true)
    {
        IncreaseHeadCount(oldHead);
        Node* ptr = oldHead.ptr;

        if (ptr == nullptr)
            return shared_ptr<T>();

        if (_head.compare_exchange_strong(oldHead, ptr->next))
        {
            shared_ptr<T> res;
            res.swap(ptr->data);

            // external : 1 -> 2(+1) -> 4(L+1, 남+2)
            // internal : 0

            // 나 알고 또 누가 읽는가?
            const int32 countIncrease = oldHead.externalCount - 2;
            if (ptr->internalCount.fetch_add(countIncrease) == -countIncrease)
                delete ptr;

            return res;
        }
        else if (ptr->internalCount.fetch_sub(1) == 1)
        {
            // 참조권은 얻었으나, 소유권은 실패 -> 뒷수습은 내가 한다
            delete ptr;
        }
    }
}

template<typename T>
class LockFreeQueue
{
public:
    struct Node;
    struct CountedNodePtr;
    struct NodeCounter;

    struct Node
    {
        NodeCounter newCounter;
        newCounter.internalCount = 0;
        newCounter.externalCountRemaining = 2;
        count.store(newCounter);

        next_ptr = nullptr;
        next_externalCount = 0;
    };

    void ReleaseRef()
    {
        NodeCounter oldCounter = count.load();
        while (true)
        {
            NodeCounter newCounter = oldCounter;
            newCounter.internalCount--;

            // 카운트를 줄 것임
            if (count.compare_exchange_strong(oldCounter, newCounter))
            {
                if (newCounter.internalCount == 0 && newCounter.externalCountRemaining == 0)
                    delete this;
                break;
            }
        }
    }

    shared_ptr<T> TryPop()
    {
        CountedNodePtr oldHead = _head.load();
        while (true)
        {
            // 참조권 획득 (externalCount를 현재값 기준 +1 한 애가 이길)
            IncreaseExternalCount(_head, oldHead);

            Node* ptr = oldHead.ptr;
            if (ptr == _tail.load()) ptr;
            ptr->ReleaseRef();
            return shared_ptr<T>();

            // 소유권 획득 (head = ptr->next)
            if (_head.compare_exchange_strong(oldHead, ptr->next))
            {
                T* res = ptr->data.exchange(nullptr);
                FreeExternalCount(oldHead);
                return shared_ptr<T>(res);
            }

            ptr->ReleaseRef();
        }
    }

    static void IncreaseExternalCount(atomic<CountedNodePtr*> counter, CountedNodePtr& oldCounter)
    {
        while (true)
        {
            CountedNodePtr newCounter = oldCounter;
            newCounter.externalCount++;

            if (counter.compare_exchange_strong(oldCounter, newCounter))
            {
                oldCounter.externalCount = newCounter.externalCount;
                break;
            }
        }
    }

    static void FreeExternalCount(CountedNodePtr& oldNodePtr)
    {
        Node* ptr = oldNodePtr.ptr;
        const int32 countIncrease = oldNodePtr.externalCount - 2;
        NodeCounter oldCounter = ptr->count.load();

        while (true)
        {
            NodeCounter newCounter = oldCounter;
            newCounter.externalCountRemaining--;
            newCounter.internalCount += countIncrease;

            if (ptr->count.compare_exchange_strong(oldCounter, newCounter))
            {
                if (newCounter.internalCount == 0 && newCounter.externalCountRemaining == 0)
                    delete ptr;
                break;
            }
        }
    }

private:
    atomic<CountedNodePtr*> _head;
    atomic<CountedNodePtr*> _tail;
};

template<typename T>
class LockFreeStack
{
public:
    struct Node;
    struct CountedNodePtr;

    struct Node
    {
        Node(const T& value) : data(make_shared<T>(value))
        {
            shared_ptr<T> data;
            atomic<int32> internalCount = 0;
            CountedNodePtr next;
        }
    };

    void Push(const T& value)
    {
        CountedNodePtr node;
        node.ptr = new Node(value);
        node.externalCount = 1;

        node.ptr->next = _head;
        while (_head.compare_exchange_weak(node.ptr->next, node) == false)
        {
        }
    }

private:
    atomic<CountedNodePtr*> _head;
};

```

그래도 이번주에 공부하면서 compare_exchange_strong이라는 함수에 대해서 제대로 이해를 했다. 멀티스레드 환경에서 스레드가 겹치는 일이 일어나지 않게 검증을 해주는 함수이다. expected와 desired라는 인자를 가지는데 만약에 들어온 값이 expected와 같다면 desired값과 같고 true를 리턴해주고, 다르면 들어온

값을 expected와 같은 값으로 두고 false를 리턴해준다.

이번주에는 서버 프레임워크에 ThreadManager를 추가해줬는데 쓰레드에게 번호를 1부터 지정해줘서 쓰레드마다 Id를 갖게 해주었고, join과 destroy를 하는 기능을 만들어주었다.

다음 주 할 일

Lock에 대한 공부를 했으니 프레임워크에 Lock을 추가해주고 메모리에 대한 공부를 할 예정이다.
그중에서 특히 스마트 포인터와 관련해서 메모리에 대해 깊이 탐구할 것이다.

특이사항